SQL SCENARIO BASED QUESTIONS

You need to retrieve the names and salaries of employees from the employees table, but only for those working in the Finance department.

Question:
How would you use the SELECT statement with a WHERE clause to retrieve specific data based on a condition?

The SELECT statement is used to query specific columns from a table and retrieve data based on certain conditions. The WHERE clause filters the rows returned by the query, ensuring only records meeting the specified criteria are included in the result set. In this case, we want only the employees working in the Finance department.

Table Structure:

```sql
-- Sample employees table structure.
CREATE TABLE employees (
    employee_id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    department VARCHAR(50),
    salary DECIMAL(10, 2)
);
```

```sql
-- Query to fetch employees in the Finance department.
SELECT first_name, last_name, salary
FROM employees
WHERE department = 'Finance';
```

This query will return rows where the department column contains the value 'Finance'. If the table contains these records:

first_name	last_name	salary
Alice	Smith	60000
Bob	Johnson	55000

The result set will display only employees belonging to Finance.

You need to add a new employee named John Doe to the employees table with a salary of 50,000 and a department of HR.

Question:
How can you use the INSERT statement to add a new record to the employees table?

The INSERT statement allows you to add a new record to a table by specifying the column names and corresponding values. You must ensure that the values match the data types and constraints (e.g., NOT NULL) defined in the table.

Table Structure:

```sql
-- Creating the employees table.
CREATE TABLE employees (
    employee_id INT PRIMARY KEY AUTO_INCREMENT,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    department VARCHAR(50) NOT NULL,
    salary DECIMAL(10, 2)
);
```

```sql
-- Inserting a new employee.
INSERT INTO employees (first_name, last_name, department, salary)
VALUES ('John', 'Doe', 'HR', 50000);
```

This query inserts John Doe into the HR department with a salary of 50,000. After insertion, the table might look like this:

employee_id	first_name	last_name	department	salary
1	John	Doe	HR	50000

You need to increase the salary of all employees in the IT department by 10%.

Question:
How would you use the UPDATE statement to modify existing records?

The UPDATE statement modifies existing rows in a table. You use the SET clause to specify the column(s) to be updated and the WHERE clause to target specific rows. In this case, we increase the salary of all employees in the IT department by 10%.

Table Structure:

```sql
-- Sample employees table.
CREATE TABLE employees (
    employee_id INT PRIMARY KEY,
    first_name VARCHAR(50),
    department VARCHAR(50),
    salary DECIMAL(10, 2)
);
```

```sql
-- Increasing IT department salaries by 10%.
UPDATE employees
SET salary = salary * 1.10
WHERE department = 'IT';
```

If the original table contains:

employee_id	first_name	department	salary
2	Alice	IT	50000
3	Bob	IT	60000

After running the query, the salaries will become:

employee_id	first_name	department	salary
2	Alice	IT	55000
3	Bob	IT	66000

The HR department has been closed, and all employees in HR must be removed from the database.

Question:
How can you use the DELETE statement to remove specific records?

The DELETE statement removes rows from a table. In this case, you must use the WHERE clause to delete only employees belonging to the HR department. Without the WHERE clause, all rows would be deleted.

```sql
-- Deleting all employees from the HR department.
DELETE FROM employees
WHERE department = 'HR';
```

If the original table contains:

employee_id	first_name	department	salary
2	Alice	IT	55000
3	Bob	HR	66000

After running the query, the table will look like:

employee_id	first_name	department	salary
2	Alice	IT	55000

You need to create a table that stores product prices, including whole numbers and decimal values.

Question:
Which data type should you use to store product prices in a table?

The DECIMAL data type is the most appropriate for storing prices since it provides high precision and control over decimal places. Using DECIMAL avoids rounding issues that might arise with floating-point data types like FLOAT.

```
-- Creating a table to store product information and prices.
CREATE TABLE products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100),
    price DECIMAL(10, 2) -- Up to 10 digits, 2 after the decimal point.
);
```

This structure ensures that prices like 123.45 or 99999.99 are stored accurately without rounding errors.

Liked these SQL questions? 🤔 Follow for more data insights, interview tips, and tech content!